BLS You:
A Zero-Query Rogue-Key Forgery Against the 2P-AISO-AS Chosen-Key Interface

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

18 October 2026

This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex generated and selected this paper's title and deliberately chose "BLS You" as BLS/"bless you" wordplay tied to the scheme name. OpenAI Codex performed AI-assisted target reconstruction, cryptanalysis, implementation, experimentation, review coordination, and technical writing. The source-backed primary reproduction returned PASS, and a fresh-context AI agent independently reproduced 32/32 attacks and all controls. A separately tasked AI adversarial reviewer attempted to disprove the result and returned `PASS_NARROW_SCOPE`, requiring every claim to remain limited to the direct-addition Definition 33 interface. Independent human verification was not performed; it is optional and is not a publication requirement.

Abstract

The security framework in IACR ePrint 2026/1394 requires its underlying two-party signature scheme with aggregatable public keys to satisfy the chosen-key experiment in Definition 33. That experiment reveals one honestly generated public key and then accepts an arbitrary second public key from the adversary before aggregating the pair. The paper's concrete BLS instantiation aggregates keys by group addition and places no proof-of-possession or key-knowledge condition on this interface.

We apply the standard BLS rogue-public-key cancellation attack. After seeing X_A, choose a known nonzero scalar β and register $X_E = \beta P - X_A$. The aggregate is βP, so for any fresh message $m^\star$ the adversary outputs $S^\star = \beta H(m^\star)$. The ordinary BLS pairing equation accepts. The attack makes zero signing-oracle queries, uses full BLS12-381 parameters, and supplies valid nonidentity subgroup elements.

An exact-verifier Rust harness imported the unchanged paper-linked artifact, used its random key generation, SHA-256 XMD hash-to-$\mathbb{G}_2$, direct G1 key addition, and ordinary pairing verifier, and accepted the forgery in 0.03 seconds with 3,072 KiB peak resident memory on one worker. Changed-message, changed-signature, honest-key-substitution, encoding, subgroup, and nonidentity controls passed. The result concerns only Definition 33's chosen-key interface. It is not a claim against Figure 6, Theorem 1, central honest KeyGen, a proper DKG, the artifact's normal lifecycle, or any deployment. The method is established prior art, and no technique novelty is claimed.

1 Introduction and exact claim

Gomez-Martinez, Tairi, Moreno-Sanchez, and Schneidewind introduce Two-Party Asymmetric-Input Solitary-Output Adaptor Signatures (2P-AISO-AS) and give a BLS-based realization in ePrint 2026/1394 [1]. Their preliminary syntax requires an underlying two-party signature scheme DS_2 with aggregatable public keys and requires it to satisfy two-party existential unforgeability. Appendix Definition 33 spells out the required experiment.

That experiment has a strong chosen-key order: the challenger generates one honest key, gives the honest verification key to the adversary, and then asks the adversary for the other verification key. The two keys are aggregated before ordinary signature verification. No proof of possession, proof of secret-key knowledge, distinct-message rule, key-prefixed message, or key-dependent coefficient appears in the interface.

This is the classic setting for a BLS rogue-key cancellation. We give the full equation, execute it through the paper-linked artifact's exact ordinary BLS verifier, and state the lifecycle boundary explicitly. Our claim is:

> Definition 33's chosen-key DS_2 interface is not existentially unforgeable when instantiated by the paper's direct-addition BLS key aggregation: a corrupt participant makes a fresh-message forgery with zero signing queries.

We do *not* claim a break of the separate Figure 6 lifecycle, Theorem 1, central honest KeyGen, a proper distributed key-generation protocol, the artifact's normal application programs, or a deployed system. The theorem is conditional on its stated underlying primitives; showing that the direct-addition BLS instantiation fails the Definition 33 assumption does not refute that conditional theorem. We also do not claim that rogue-key cancellation is new. The target-specific result is that the paper expressly requires Definition 33 while its concrete direct-addition BLS interface admits the standard attack.

2 Target and evidence identity

The target is the retained 83-page PDF of *Adaptor Signatures Meet BLS: Enabling Efficient Blockchain Applications with Unique Adaptor Signatures*, IACR ePrint 2026/1394, received 8 July and approved 12 July 2026. Its SHA-256 digest is

`93c36cf110ab278be767c367c20ae2035446c1bbffc3e1429551d8b42e473e4f.`

The exact text reviewed for Definition 33 has SHA-256

`890fab6e15b5276537881eaefa49e8224226dd3933ce7e018e38ba0e560a5001.`

The paper links an Anonymous GitHub artifact [2]. The retained official ZIP has SHA-256

`f1e70afef59a55e3a852d1a9debfefb7f9250f90679a65faa46ecdf0c0a7b441.`

It contains 65 files and 178,502 uncompressed bytes. The mirror exposes no Git repository identity, branch, commit, or tree, so the ZIP digest and the complete per-file manifest are the authoritative source pin. We make no Git revision claim. The relevant unchanged files include:

File	SHA-256
`bls-aiso-as/src/bls.rs`	`f26362cdb3eb14485e62a6fd7085ffb58cdd0597013f52a39b260de7d49de52e`
`bls-aiso-as/src/common.rs`	`cd7a2d0498f1f62ed8679e70440c9c3b2057f3cd8094ebd5d87d557a1c188344`
`test-as/src/main.rs`	`bef2c4fb6a0f6b2de5fbdbf4fdf390a58ea81daa0a4c6bc4fbd0ef8b67b50d12`
`test-as-optimized/src/main.rs`	`e33790341c5ad68095dcb6808b7a0232938d49edf3104a81034faeb57889dadf`

The artifact uses the BLS12-381 implementation distributed as `irmaseal-curve` 0.1.4. Public keys are G1 points, signatures are G2 points, and messages are hashed to G2 using SHA-256 XMD with domain string `BLS12-381-G2:SHA-256`. The ordinary verifier in `bls.rs`, lines 110–114, is the endpoint exercised below.

3 Definition 33 and the BLS endpoint

Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ have prime order r, let P generate $\mathbb{G}_1$, and let

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \longrightarrow \mathbb{G}_T$$

be a nondegenerate bilinear pairing. Let $H : \{0,1\}^* \to \mathbb{G}_2$ be the artifact's hash-to-curve function. In the artifact's orientation, a secret $x \in \mathbb{Z}_r$ has public key $X = xP$, and a signature on m is $S = xH(m)$. Ordinary verification accepts exactly when

$$e(P, S) = e(X, H(m)). \tag{1}$$

The concrete two-key aggregation used by the artifact test programs is direct G1 addition:

$$\mathsf{KAg}(X_A, X_E) = X_A + X_E. \tag{2}$$

Definition 33 first initializes an empty query set Q, generates the honest keypair, and reveals X_A to the adversary. Only then does it accept the adversarial key X_E. After an optional signing-oracle phase, it computes $\bar{X} = \mathsf{KAg}(X_A, X_E)$ and accepts an adversarial pair $(m^\star, S^\star)$ when $m^\star \notin Q$ and Equation (1) holds under $\bar{X}$. The attack below never invokes the signing oracle, so Q remains empty.

4 Zero-query rogue-key forgery

Theorem 1. *The direct-addition BLS instantiation fails Definition 33 with probability one, apart from avoidable choices of an identity key, using zero signing queries and constant group work.*

Proof. After receiving the honest public key X_A, choose any known $\beta \in \mathbb{Z}_r \setminus \{0\}$ for which $\beta P \neq X_A$, and return

$$X_E = \beta P - X_A. \tag{3}$$

The condition is publicly testable. If one candidate produces the identity, select another nonzero β. Because X_A and βP are elements of the prime-order subgroup and that subgroup is closed under subtraction, X_E is a valid subgroup element. The explicit inequality makes X_E nonidentity. Equation (2) now gives

$$\bar{X} = X_A + X_E = \beta P, \tag{4}$$

which is also nonidentity because $\beta \neq 0$.

Choose any fresh message $m^\star$ and output

$$S^\star = \beta H(m^\star). \tag{5}$$

The adversary makes no signing query. Bilinearity and Equation (4) give

$$e(P, S^\star) = e(P, \beta H(m^\star)) = e(\beta P, H(m^\star)) = e(\bar{X}, H(m^\star)),$$

so the ordinary verifier accepts. The adversary knows the aggregate secret β but need not know the discrete logarithm $\beta - x_A$ of X_E. Definition 33 asks only for the public key and contains no knowledge check. □

The work is one scalar multiplication and one subtraction in G1 to construct the corrupt key, one hash to G2, and one scalar multiplication in G2 to forge. Memory is constant. There is no search, reduced parameter, collision, preimage, malformed encoding, parser behavior, weak honest key, fault, side-channel, or implementation bug.

5 Source-backed execution and controls

The primary Rust harness imports the unchanged `bls-aiso-as` crate. It calls `BLSKeyPair::new()` for the honest key, selects $\beta = 42$ (or 43 in the publicly detected identity edge case), constructs Equation (3), applies direct G1 addition, calls the artifact's `hash_to_g2`, and sends Equation (5) to the public `bls::verify`. It contains no call to a signing function.

The full BLS12-381 execution returned PASS. The direct attack run took 0.03 seconds and 3,072 KiB maximum resident memory; a sequential repeat took 0.04 seconds with the same memory and byte-identical standard output. Both used one worker and a 1 GiB virtual-memory limit. A clean locked offline build-and-run took 1.35 seconds and 120,320 KiB. The source output recorded `kag_equals_beta_g1=true`, zero signing queries, a fresh message, and acceptance by the ordinary artifact verifier.

The harness also establishes:

- the honest, corrupt, and aggregate G1 keys are nonidentity and torsion-free;
- the forged signature is computed in the G2 prime-order subgroup;
- changing the message rejects;
- adding the G2 generator to the signature rejects;

- replacing the honest key rejects; and
- malformed all-`ff` compressed G1 and G2 strings are rejected by the concrete checked decoding boundary.

A separately written fresh-context harness ran 32 randomized honest-key trials through the same unchanged verifier and returned 32/32 attack successes with all changed-message, changed-signature, honest-key-substitution, nonidentity, subgroup, checked-encoding, and malformed-encoding controls passing. Its cryptographic checks took 1.16381577 seconds; the clean locked compile-and-run took 102.98 seconds wall and 400,048 KiB maximum resident memory with one worker. The sealed `REVIEW_ARTIFACTS.sha256` file has SHA-256 `b9c45ff8afea908c2732690215ad80f5acb0e67b4c49567cb304c449ff798a8c`; the state handoff has SHA-256 `b089529d0146daeb37bd50f370411c311012283a25d1ca090aaaf128eb67d22e`.

The sealed post-evidence adversarial review returned `PASS_NARROW_SCOPE`. It independently checked the key-choice order, validity and ownership distinction, aggregation semantics, unchanged verifier, freshness, controls, practical resources, profile pin, prior-art attribution, and lifecycle boundary. Its manifest has SHA-256 `f4558bb0b73c6e0a6ae7c1b8b400ebdc2f3cabde0ec71f93cbc7c01ba54eb065`, and its handoff has SHA-256 `b7218c2be45c0683a1243a639e7be3d510db30018efa230ccaefb6d226b6a2db`.

6 Lifecycle boundary

The relevant interfaces must remain separate:

Interface	Key provenance	Claim
Definition 33	Honest X_A is revealed before the adversary chooses X_E; no proof of possession	Broken by the zero-query forgery
Figure 6	Both participant keypairs come from central honest KeyGen	Excluded from this claim
Theorem 1	Conditional construction theorem whose assumptions include the required security of the underlying DS_2	Not refuted by this result
Artifact programs	Both keypairs are independently generated with `BLSKeyPair::new()`	Excluded from this claim
Proper DKG or authenticated registration	Requires a separately specified knowledge and binding guarantee	Excluded and not evaluated
Deployments	Deployment-specific registration and lifecycle rules	No deployment claim

Figure 6 and the normal artifact programs do not let the corrupt participant replace its key after seeing the honest key. Footnote 7 describes central key generation as a syntactic simplification that could be realized by DKG. That is materially different from Definition 33. The source execution is therefore an exact-verifier diagnostic for the required chosen-key interface, not an end-to-end artifact-lifecycle demonstration.

This boundary does not make the Definition 33 result merely an implementation observation. The interface is printed as the required security notion for the underlying DS_2, and the cancellation follows from its key order and the concrete aggregation equation. It does mean that every title, abstract, table, and summary must name Definition 33 rather than saying without qualification that all of 2P-AISO-AS or a deployment is broken.

7 Prior art and history search

The method is established. Boneh, Gentry, Lynn, and Shacham discuss BLS aggregate signatures and the chosen-public-key problem, using distinct messages or public-key-prefixed inputs as a defense [3]. Bellare and Neven formalize multisignatures in the plain public-key model, where malicious keys may depend on

honest keys [4]. Boneh, Drijvers, and Neven give the same cancellation explicitly: choose a key equal to a known generator power times the inverse of the honest key, then sign under the known aggregate exponent [5]. The current CFRG BLS draft describes Basic, Augmentation, and Proof-of-Possession variants and warns that fast common-message aggregate verification requires a valid proof for every public key [6].

Against the Definition 33 chosen-key DS_2 interface of ePrint 2026/1394, we apply the standard BLS rogue-public-key cancellation attack and obtain a zero-query fresh-message forgery at full BLS12-381 parameters. We located no earlier public report applying this attack to that exact interface in searches completed through 18 October 2026. The attack technique is prior art; no technique novelty is claimed. This is a dated, bounded search result and does not exclude private, deleted, embargoed, unindexed, or later disclosures.

The final history recheck returned `PASS_FINAL_HISTORY`. Its bundle manifest has SHA-256 `796daa3485222c38b14b3e6c6b2e6c7a4514df09efb60c8bbb31b67ada0e25f3`; the 74-source manifest has SHA-256 `a12410ab6c67aee1d3a2fbd9caf2acc2667007ccfaa4d1512f23ebfd061f9e95`; and the handoff has SHA-256 `847ad6ca5617a2491728e178b957cdc2a096cf6beb9ba0a9c6c7919f072481e3`.

8 Repair directions and limitations

Public-key subgroup and nonidentity checks are necessary but do not stop this attack: X_E already passes them. Standard design directions include:

- require and verify a proof of possession or proof of secret-key knowledge before accepting either contribution;
- augment each signed message with the corresponding public key;
- use list-bound, key-dependent aggregation coefficients as in the BDN line of work; or
- replace the chosen-key interface with authenticated honest generation or a DKG that proves possession and binds every contribution.

Each option changes an interface or equation used by the paper. Its fit with the two-party protocol, adaptor operations, uniqueness properties, and proof must be analyzed afresh. This audit has not implemented or validated a repair.

The evidence has the following limits:

- the claim is exactly the Definition 33 chosen-key DS_2 interface;
- Theorem 1, Figure 6, and the normal artifact key lifecycles are outside the attack claim;
- the artifact exposes typed group points rather than a public adversarial byte-registration API;
- the anonymous mirror provides a hash-pinned ZIP but no Git commit;
- the history statement is bounded to the public sources searched through 18 October 2026; and
- all required technical, adversarial, independent-reproduction, final-history, and publication checks were completed before classification.

9 Conclusion

Definition 33 reveals an honest BLS key before accepting an unproved corrupt key and then aggregates by direct addition. Standard rogue-key cancellation therefore makes the aggregate key's discrete logarithm known to the adversary. A fresh BLS signature follows with zero signing queries, and the unchanged full-parameter artifact verifier accepts it. This result is precise but narrow: it breaks the printed chosen-key interface, while Theorem 1 and every honest-key or proper-DKG lifecycle remain outside the attack claim.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

[1] J. Gomez-Martinez, E. Tairi, P. Moreno-Sanchez, and C. Schneidewind. Adaptor Signatures Meet BLS: Enabling Efficient Blockchain Applications with Unique Adaptor Signatures. IACR Cryptology ePrint Archive, Report 2026/1394, 2026. https://eprint.iacr.org/2026/1394.

[2] Gomez-Martinez et al. Paper-linked 2P-AISO-AS artifact, source snapshot dated 7 May 2026. https://anonymous.4open.science/r/Adaptor_Signatures_Meet_BLS-ECD0/.

[3] D. Boneh, C. Gentry, B. Lynn, and H. Shacham. Aggregate and Verifiably Encrypted Signatures from Bilinear Maps. In *EUROCRYPT 2003*, LNCS 2656, pages 416–432, 2003. https://eprint.iacr.org/2002/175.

[4] M. Bellare and G. Neven. Multi-signatures in the Plain Public-Key Model and a General Forking Lemma. In *ACM CCS 2006*, pages 390–399, 2006.

[5] D. Boneh, M. Drijvers, and G. Neven. BLS Multi-Signatures With Public-Key Aggregation. 2018. https://crypto.stanford.edu/~dabo/pubs/papers/BLSmultisig.html. See also IACR ePrint 2018/483, *Compact Multi-Signatures for Smaller Blockchains*.

[6] D. Boneh et al. BLS Signatures, Internet-Draft `draft-irtf-cfrg-bls-signature-07`, 6 July 2026. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bls-signature/.